```
            // The original thread needs to join the two
threads we create.  That
              // way this main thread does not end (and
terminate the program) until
            // t1 and t2 are done.
            t1.Join();
            t2.Join();
            Console.ReadLine();
        }
        // Our static counter, shared by t1 and t2
        static int counter = 0;

        static void PrintSeq()
        {
            // t1 and t2 each have their own copy of i
            for (int i = 0; i < 100; i++)
            {
              Console.WriteLine(Thread.CurrentThread.Name +
                                ": {0}, {1}", i, counter++);
            }
        }
    }
```

There is not a straightforward way for threads to share local data because they each have their own copies. However, threads can both read and write to shared data on the heap as shown above with the counter variable. This should raise a red flag in your head: can conflicts arise between threads reading and writing from the same share variable? Definitely!

Here is the beginning of a sample run of BasicThreadExample:

```
t1: 0, 0
 t1: 1, 2
 t1: 2, 3
 t1: 3, 4
 t1: 4, 5
 t2: 0, 1
 t2: 1, 7
 t2: 2, 8
```